Parsing requests & writing responses

9 min read · Updated 24 Jul 2026 · Topic 31 of 70

Bookmark · Mark done

Why it matters

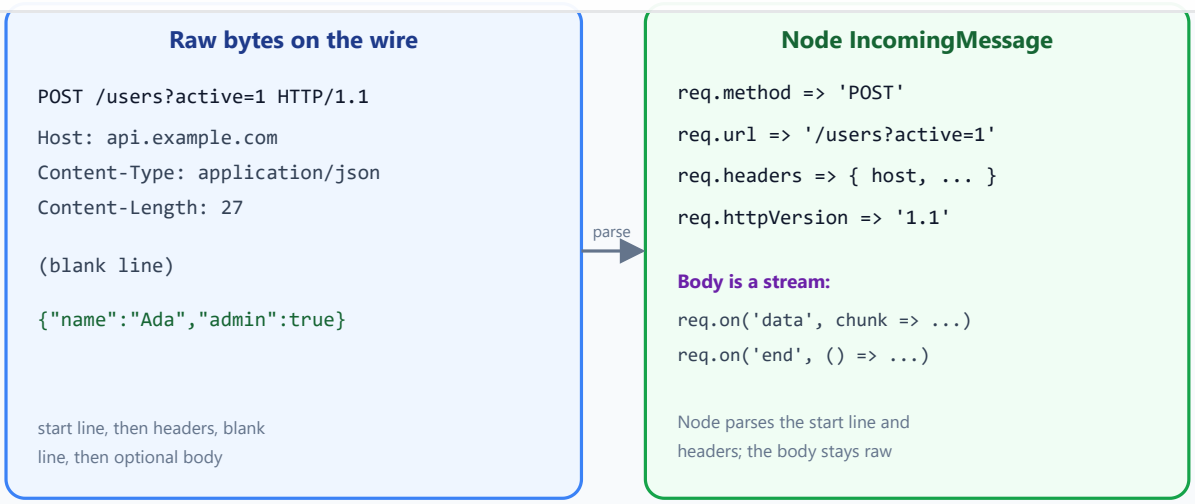
The `req` and `res` objects are where your server meets the network. Everything a client tells you — which resource, which verb, what content type, the auth token, the JSON payload — arrives on `req`. Everything you tell the client back — success or failure, the shape of the data, caching rules, cookies — you encode into `res`. Get these two objects right and your endpoints are correct, cacheable, and secure; get them wrong and you leak memory on large uploads, corrupt UTF-8 text, or silently accept malformed input.

This is also the layer frameworks hide, which means when something misbehaves — a body that never parses, a response that arrives with the wrong charset, a header the browser rejects — the fix lives here. And in interviews, "how do you read a POST body without Express?" separates people who understand streams from people who have only memorised `app.use(express.json())`.

The concept, simply

An HTTP request has four parts: a **method** (GET, POST…), a **target** (the URL path plus query string), a set of **headers** (key/value metadata), and an optional **body** (the payload). Node parses the first three for you and exposes them synchronously as `req.method`, `req.url`, and `req.headers`. The body it does *not* parse — it hands you a stream, because a body can be gigabytes and Node will not silently buffer that into memory for you.

A response mirrors this: a **status code** (200, 404, 500…), **headers**, and a **body**. You set the status and headers, then write the body. That is the whole contract.



Node parses the start line and headers into properties; the body stays a raw stream.

How it works

Method and URL. `req.method` is the uppercase verb string. `req.url` is only the *path and query* — e.g. `'/users?active=1'` — not the full URL; the host lives in `req.headers.host`. To parse it safely, use the WHATWG `URL` class with a base: `new URL(req.url, \`http://${req.headers.host}\`)`, which gives you `.pathname` and a `.searchParams` object. The legacy `url.parse()` is deprecated and has security footguns.

Headers. `req.headers` is a plain object with **lowercased** keys (HTTP header names are case-insensitive, so Node normalises them). Most headers are strings; `set-cookie` is special and is always an array. Duplicate headers other than a few are joined with `', '`. There is also `req.rawHeaders`, a flat array preserving original case and order, if you need the wire form.

The body. Because `req` is a readable stream, you assemble the body by collecting chunks. Chunks are `Buffer`s, not strings — concatenate the buffers and decode *once* at the end, otherwise a multi-byte UTF-8 character split across two chunks gets corrupted. You must also **size-limit** the body: an attacker (or a bug) can stream forever, so abort past a threshold. In modern Node you can skip the manual loop entirely with `req.text()` / `req.json()`-style helpers via `node:stream/consumers`, but knowing the manual version is expected.

`res.setHeader(name, value)` sets one header (and can be called repeatedly before the body). The body is written with `res.write()` (0+ times) and finished with `res.end()`. Always send a `Content-Type` — without it browsers guess ("MIME sniffing"), which is both a correctness and a security problem. For text, include the charset: `text/plain; charset=utf-8`.

Content-Length vs chunked. If you know the full body size, set `Content-Length` and the response is sent with that length. If you stream with multiple `write()`s and no length, Node uses *chunked transfer-encoding* automatically. Both are valid; you rarely set the encoding by hand.

Backpressure on the response. `res.write(chunk)` returns a boolean: `true` means the chunk fit in the kernel/socket buffer, `false` means the buffer is full and the data is being queued in memory. If you are copying from a fast source (a file, a database cursor) to a slow client and you ignore that `false`, Node keeps buffering and memory climbs unbounded. The correct pattern is to stop writing, wait for the writable stream's `'drain'` event, then resume — or simply `pipeline(source, res)` and let Node manage the flow control for you. This is the single most important reason to prefer `pipe`/`pipeline` over a hand-rolled read-and-write loop when relaying a stream to a response.

What ends a response, and what a 1xx or HEAD changes. A response is complete when Node has sent everything the framing promises: a full `Content-Length` worth of bytes, or the terminating zero-length chunk for chunked encoding. `res.end()` writes that terminator. Two edge cases surprise people. For a `HEAD` request you must send the exact same headers (including `Content-Length`) as the equivalent `GET`, but no body — so call `res.end()` with no data. And informational `1xx` responses (like `100 Continue` or `103 Early Hints`) are sent with `res.writeProcessing()` / `res.writeEarlyHints()` *before* the final status; they do not count as "the" response and do not lock your headers, so you can still send a normal `200` afterwards.

Trailers. Occasionally you need to send a header whose value you only know *after* the body (a checksum, a computed length). HTTP supports *trailers* for exactly this: declare them up front with `res.setHeader('Trailer', 'Content-MD5')` (only valid with chunked encoding), stream the body, then supply the values with `res.addTrailers({`

Code examples

Reading a JSON body safely, with a size cap and correct decoding:

JAVASCRIPT Copy

```
const http = require('node:http');

const MAX_BODY = 1_000_000; // 1 MB

function readJson(req) {
  return new Promise((resolve, reject) => {
    const chunks = [];
    let size = 0;
    req.on('data', (chunk) => {
      size += chunk.length;
      if (size > MAX_BODY) {
        reject(Object.assign(new Error('Payload too large'), { status: 413
        req.destroy();
        return;
      }
      chunks.push(chunk);
    });
    req.on('end', () => {
      if (chunks.length === 0) return resolve(undefined);
      try {
        resolve(JSON.parse(Buffer.concat(chunks).toString('utf8')));
      } catch {
        reject(Object.assign(new Error('Invalid JSON'), { status: 400 }));
      }
    });
    req.on('error', reject);
  });
}

http.createServer(async (req, res) => {
  if (req.method === 'POST') {
    try {
      const body = await readJson(req);
      res.writeHead(200, { 'Content-Type': 'application/json' });
```

```
      res.end(JSON.stringify({ error: err.message }));
    }
  } else {
    res.writeHead(405, { Allow: 'POST' }).end();
  }
}).listen(3000);
```

The same body read using the modern stream-consumers helper (Node 16.7+), which handles the loop for you:

JAVASCRIPT Copy

```
const { json } = require('node:stream/consumers');

http.createServer(async (req, res) => {
  const body = await json(req); // reads + JSON.parses the stream
  res.setHeader('Content-Type', 'application/json');
  res.end(JSON.stringify({ received: body }));
}).listen(3000);
```

Inspecting the request — method, path, query, and a header — and echoing them back:

JAVASCRIPT Copy

```
http.createServer((req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  const info = {
    method: req.method,
    path: url.pathname,
    query: Object.fromEntries(url.searchParams),
    userAgent: req.headers['user-agent'] ?? null,
    httpVersion: req.httpVersion,
  };
  res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' }
  res.end(JSON.stringify(info, null, 2));
}).listen(3000);
```

Setting several response headers, including a cookie, and choosing an explicit status:

Node.js Backend Kit

JAVASCRIPT

Copy

```
http.createServer((req, res) => {
  res.statusCode = 201;
  res.setHeader('Content-Type', 'application/json');
  res.setHeader('Cache-Control', 'no-store');
  res.setHeader('Set-Cookie', 'sid=abc123; HttpOnly; Path=/; SameSite=Lax'
  res.end(JSON.stringify({ id: 42 }));
}).listen(3000);
```

Common mistakes & gotchas

- **Treating `req.url` as the full URL.** It is only the path + query. Building a `URL` without a base throws; supply `http://${req.headers.host}`.
- **Decoding each chunk separately.** `chunk.toString()` per chunk corrupts multi-byte characters that straddle a chunk boundary. Concatenate buffers, then decode once.
- **No body size limit.** Buffering an unbounded stream into an array is a denial-of-service vector. Always cap and abort.
- **Assuming header values are strings.** `set-cookie` is always an array; some proxies send repeated headers. Handle both.
- **Omitting `Content-Type`.** Leads to MIME sniffing and broken clients; for JSON always send `application/json`.
- **Wrong status semantics.** Returning `200` with an error body, or `500` for user input errors. Use `4xx` for client faults, `5xx` for server faults.
- **Trusting header case.** Reading `req.headers['Content-Type']` returns `undefined` — keys are lowercased. Use `'content-type'`.

Best practices

- **Parse URLs with the WHATWG `URL` class**, never the deprecated `url.parse()`.
- **Cap body size** and reject with `413 Payload Too Large` before buffering the rest.

claimed was `text/html`.

- **Return the right status code**: `400` invalid, `401` unauth, `404` missing, `405` wrong method (with an `Allow` header), `413` too big, `500` server error.
- **Set security headers** (`X-Content-Type-Options: nosniff`, `Cache-Control`) deliberately rather than relying on defaults.
- **Prefer** `node:stream/consumers` (`json()`, `text()`, `buffer()`) for concise, correct body reading.

⭐ Interview lens

Q: How do you read a POST body in raw Node?
Listen on the `req` stream: collect `Buffer` chunks in `'data'`, concatenate them in `'end'` with `Buffer.concat`, then decode with `.toString('utf8')` and parse. Enforce a max size and handle `'error'`. Modern shortcut: `await require('node:stream/consumers').json(req)`.

Q: Why concatenate buffers instead of building a string chunk by chunk?
A UTF-8 character can span two chunks; calling `toString()` on each chunk decodes a partial byte sequence and produces replacement characters (`&#xFFFD;`). Concatenating the raw buffers first and decoding once preserves the bytes.

Q: What is in `req.url` exactly?
Only the request target: the path and query string (e.g. `/search?q=node`). Not the scheme, not the host. The host is in `req.headers.host`. To parse, use `new URL(req.url, `http://${req.headers.host}`)`.

Q: Trap — why does `req.headers['Content-Type']` come back `undefined`?
Node lowercases all header keys, so the correct access is `req.headers['content-`

Q: What's the difference between `writeHead` and `setHeader`?
`setHeader(name, value)` sets one header and can be called repeatedly before the body; the header can still be read back or overwritten. `writeHead(status, headers)` sets the status and a batch of headers and marks them as sent — after it, headers are effectively flushed. You cannot `setHeader` after `writeHead`/first `write`.

Q: Which status for "the client used the wrong HTTP method"?
`405 Method Not Allowed`, and you must include an `Allow` header listing the accepted methods. Returning `404` or `400` is a common wrong answer.

Cheat-card

- `req.method` — uppercase verb; `req.url` — path + query only.
- `req.headers` — lowercased keys; `set-cookie` is an array.
- Body = stream: `'data'` (Buffers) → `Buffer.concat` → `.toString('utf8')`.
- Parse URL: `new URL(req.url, `http://${req.headers.host}`)`.
- Shortcut: `stream/consumers` → `json()` / `text()` / `buffer()`.
- Response: set status + headers → `write` → `end`.
- Always send `Content-Type`; cap body size (413).
- Status map: 400 / 401 / 404 / 405+Allow / 413 / 500.

QUICK CHECK — no marks, just for you.

1. Why should you concatenate the raw body Buffers and decode once, rather than calling chunk.toString() per chunk?

- toString() only works on the first chunk of a request
- A multi-byte UTF-8 character can span two chunks, so decoding each chunk separately corrupts it into replacement characters
- Concatenating buffers is required because chunks arrive out of order

Check answer

2. What exactly does req.url contain in a raw Node HTTP server?

- The full absolute URL including protocol and hostname
- Only the path, with the query string stripped out
- The hostname only; the path lives in req.headers.host
- Only the request target: the path and query string (e.g. /search?q=node), not the scheme or host

Check answer

3. Which status code should you return when a client uses an HTTP method a known path does not support?

- 405 Method Not Allowed, including an Allow header listing the accepted methods
- 404 Not Found, since the specific request could not be served
- 400 Bad Request, because the client made a mistake
- 403 Forbidden, since the method is not permitted

Previous
The http module & createServer

Next
Manual routing & serving JSON

LEARN MORE

ARTICLES & DOCS

- HTTP messages | MDN
  developer.mozilla.org
- Overview of HTTP | MDN
  developer.mozilla.org
- HTTP | Node.js Documentation
  nodejs.org

VIDEOS



How to Create a Simple HTTP Server in Node.js | HTTP Module Basics
ProgrammingKnowledge

Hand-picked references for this lesson.